

- *isOpen()* checks whether a connection is active or not.
- *transaction()* begins a transaction if supported by the database and driver.
- *commit()* commits any pending operation, if any, in the present transaction.
- *rollback()* rolls back a transaction if it fails.
- *lastError()* returns an object of *QSqlError* if an error occurs.
- *connectionName()*, *connectOptions()*, *databaseName()*, *driverName()*, *hostName()*, *userName()*, *password()*, *port()*, *connectionNames()*, *drivers()* returns details of the connection, database, driver, etc.
- *setConnectOptions(opts)* sets database-specific options.
- *tables()* returns a list of tables that exist in the present database in the form of *QStringList*.
- *record(tableName)* returns the *QSqlRecord* object with filed names (headers) in a specified table.

QSqlQuery

This class is responsible for executing and manipulating SQL statements. It takes a connection object as an optional argument in the constructor.

- *prepare* prepares a query with place holders. The place holder can be name-based or position based.
- *bindValue* binds the actual data to place holders.

```
query.prepare("update student set marks=? Where rollno=?");
query.addBindValue(90.23); query.addBindValue(105);
// query.bindValue(0,90.23); query.bindValue(1,105); can be used instead
...or:
query.prepare("update student set marks=:marks where rollno=:rno");
query.bindValue(":marks", 90.23); query.bindValue(":rno", 105);
```
- *exec* executes a prepared query.
- *exec(query)* executes a new query passed as a string, or you can pass this string to the constructor itself, which will be executed automatically. Once the query is executed you can use the navigation functions *first()*, *previous()*, *next()*, and *last()* to place the cursor on the first, previous, next, and last records, respectively.
- *seek(index, relative=false)* places the cursor on a record specified by the index. An absolute or relative positioning will be used based on the second argument.
- *at()* returns the present position of the cursor.
- *setForwardOnly(mode)* is to allow only forward navigation functions.
- *record()* returns the record at the current position as an object of *QSqlRecord*.
- *result()* returns the *QSqlResult* object for a given query.
- *value(index)* returns data of a field specified by the index in the form of *QVariant*.
- *size()* returns the number of rows in the result set of queries.
- *numRowsAffected()* returns the number of rows affected by the query.
- *isValid()* tells whether the cursor is on the valid record.
- *isSelect()* tells whether the current query is a SELECT

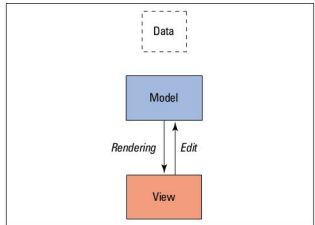


Figure 1: The model/view architecture

[Source: Qt Documentation]

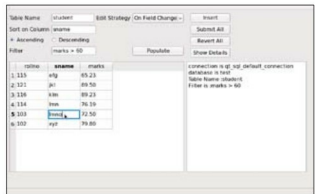


Figure 2: An editable model populated with data from the student table

operation or not.

- *lastQuery()* returns the last query executed as a string, or null, if none is executed.
- *executedQuery()* returns the last query executed successfully as a string.
- *lastError()* returns an object that represents the error state of a recent operation.

Model/view programming in Qt

The model/view architecture (see Figure 1) is a slight variation from the MVC architecture. It is achieved by combining the view and controller together. Only two layers exist here—one to communicate with the data, and the other for the user interface and interaction. Optionally, you can use delegates in between to customise the way data items are rendered and edited in a better way.

Qt comes with a set of widgets to act as models and views. Basically, three kinds of views are available, all inherited from *QAbstractItemView*. These are:

- List View
- Tree View
- Table View

Some other views like Graphics View and Web View come with specific modules. You can combine many standard models with these views. For example, you can